

ScholarConnect — System Architecture Documentation

ScholarConnect

Complete Architecture & Engineering Design Package
Generated: April 2026

Table of Contents

Section A — System Architecture

- 1. High-level system architecture
- 2. Client-server communication
- 3. Component architecture

Section B — Data Flow Diagrams

- 4. DFD Level 0 (Context)
- 5. DFD Level 1
- 6. DFD Level 2 (Critical Subsystems)

Section C — Database / Data Model

- 7. ER Diagram (Logical)
- 8. Database Schema / Collections
- 9. Schema Design Document

Section D — Algorithm & Workflows

- 10. Skill Selection Flow
- 11. Project Recommendation Flow
- 12. Mentor Matching Flow
- 13. Smart Invite / Collab Matching
- 14. AI Copilot / Generative Flow
- 15. GitHub Import Pipeline
- 16. Collaboration Lifecycle

Section E — User Flows

- 17. End-to-end User Journey
- 18. Persona Flows (Mentee, Mentor, Admin)
- 19. State Transitions

Section F — Advanced Engineering Diagrams

- 20. Advanced Diagrams

Section G — Security & Scalability

- 21. Security Architecture
- 22. Scalability Architecture

Section A

System Architecture

1. High-Level System Architecture

ScholarConnect is a distributed, service-oriented architecture comprised of a React frontend, Node.js/Express core backend, and a specialized Python machine-learning service. The entire platform is backed by a relational database (SQLite in development, configurable for PostgreSQL/MySQL in production).

```
````mermaid graph TD
 subgraph Client_Layer
 Web[React Web App]
 Mobile[Mobile Responsive View]
 end
```

```
 subgraph API_Gateway_Core_Backend
 Gateway[Express Server Node.js]
 AuthSvc[Auth & Identity Service]
 CollabSvc[Collaboration & Workspace Service]
 AdminSvc[Admin Services]
 end
end
```

```
 subgraph AI_ML_Service_Python
 RecEngine[Recommendation Engine]
 MatchEngine[Matching Engine Mentors/Teammates]
 NLP[NLP & Copilot Processing]
 end
end
```

```
 subgraph Data_Persistence
 DB[(Relational DB\nSQLite/PostgreSQL)]
 end
```

end

```
subgraph External Integrations
 GitHub[GitHub API]
 GoogleAuth[Google OAuth]
 GitHubAuth[GitHub OAuth]
end
```

Web -->|REST API| Gateway  
Mobile -->|REST API| Gateway

Gateway --> AuthSvc  
Gateway --> CollabSvc  
Gateway --> AdminSvc

AuthSvc --> GoogleAuth  
AuthSvc --> GitHubAuth  
CollabSvc --> GitHub

Gateway -->|HTTP POST JSON| RecEngine  
Gateway -->|HTTP POST JSON| MatchEngine  
Gateway -->|HTTP POST JSON| NLP

Gateway --> DB

```
classDef client fill:#3b82f6,stroke:#2563eb,color:#fff;
classDef backend fill:#10b981,stroke:#059669,color:#fff;
classDef ml fill:#8b5cf6,stroke:#7c3aed,color:#fff;
classDef data fill:#f59e0b,stroke:#d97706,color:#fff;
classDef ext fill:#64748b,stroke:#475569,color:#fff;
```

```
class Web,Mobile client;
class Gateway,AuthSvc,CollabSvc,AdminSvc backend;
class RecEngine,MatchEngine,NLP ml;
class DB data;
class GitHub,GoogleAuth,GitHubAuth ext;
```

<div class="diagram-caption">Figure 1: High-level System Architecture</div>  
</div>

### 2. Client-Server Communication Architecture

The communication between the client and server strictly follows RESTful principles over HTTP. Stateful communication (such as tracking real-time collaboration) relies on persistent polling or Server-Sent Events (SSE) depending on configuration, backed by stateless JWT authentication for every request.

<div class="diagram-container">

```
```mermaid
sequenceDiagram
    participant C as Client (React)
    participant M as Auth Middleware
    participant B as Core Backend (Node)
    participant ML as ML Service (Python)
    participant D as Database
```

```
C->>B: POST /api/auth/login
B->>D: Verify Credentials
D-->>B: User Record
B-->>C: Returns JWT & User Data
```

```
Note over C,D: Subsequent Requests
C->>M: GET /api/projects/recommend (Bearer JWT)
M->>M: Verify Token
M->>B: Route to Controller
B->>D: Fetch User Profile & Skills
D-->>B: Profile Data
```

```
B-->>ML: POST /recommend (Profile)
ML-->>B: Recommended Project Array
B-->>C: Returns 200 OK (Projects)
```

Figure 2: Request/Response Flow Diagram

3. Component Architecture

ScholarConnect follows a strict modular structure. Components are decoupled by domain.

```
```mermaid graph LR
 subgraph Frontend Modules
 Dashboard[Dashboard]
 MentorMatchingUI[Mentor Matching]
 TeammateMatchingUI[Teammate Matching]
 WorkspaceUI[Collab Workspace]
 AdminUI[Admin Dashboard]
 end
```

```
subgraph Backend Services Node.js
 CollabAPI[Collaboration API]
 AuthAPI[Auth Services]
 WorkspaceAPI[Workspace & Chat]
 AdminAPI[Admin Services]
end
```

```
subgraph ML Microservice Python
 RecEngine[Project Recommender]
 MentorMatch[Mentor Matcher]
 TeammateMatch[Teammate Matcher]
 GenAI[Generative Co-pilot]
end
```

```
Dashboard --> RecEngine
MentorMatchingUI --> MentorMatch
TeammateMatchingUI --> TeammateMatch
WorkspaceUI --> WorkspaceAPI
WorkspaceUI --> CollabAPI
AdminUI --> AdminAPI
```

```
WorkspaceAPI -. -> GenAI
```

```
style Frontend Modules fill:#f8fafc,stroke:#e2e8f0
style Backend Services Node.js fill:#f0fdf4,stroke:#bbf7d0
style ML Microservice Python fill:#faf5ff,stroke:#e9d5ff
```

```
<div class="diagram-caption">Figure 3: Internal Component Service Boundaries</div>
</div>

</div>
```

```
<div class="page">
```

```
<div class="section-label">Section B</div>
<h2>Data Flow Diagrams (DFD)</h2>
```

```
<h3>4. DFD Level 0 (Context Diagram)</h3>
```

```
<p>The Level 0 Context Diagram establishes the boundaries of the ScholarConnect system, showing external entities and the major data flows into and out of the system.</p>
```

```
<div class="diagram-container">
```

```
```mermaid
graph TD
    User([Student / User])
    Mentor([Mentor / Faculty])
    Admin([Administrator])
    GitHub([GitHub Platform])
    OAuth([OAuth Provider])

    System[ScholarConnect Platform]
```

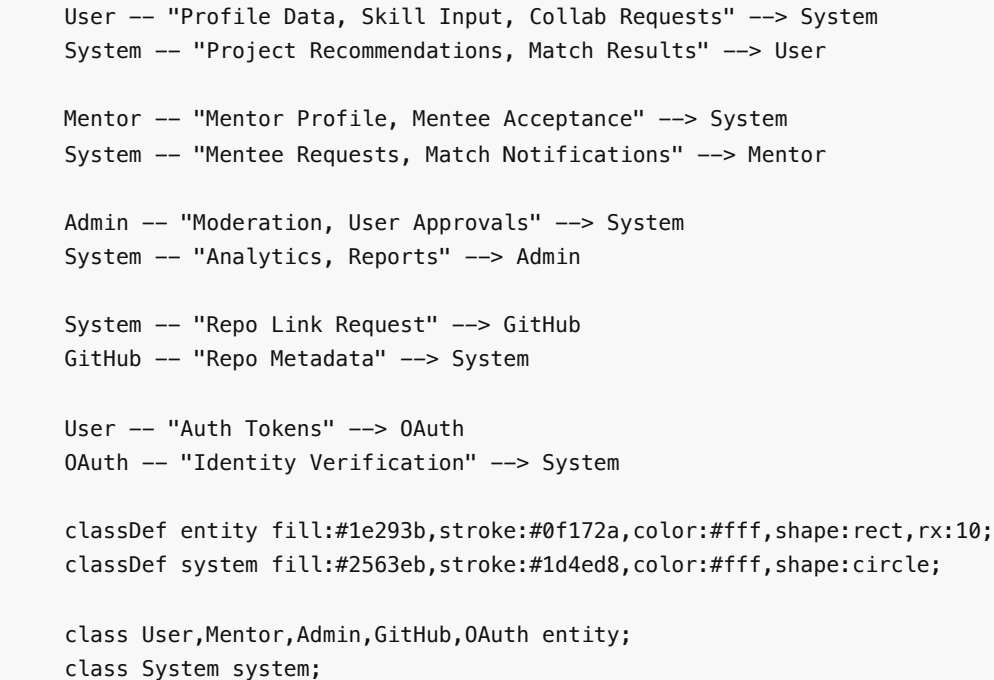


Figure 4: Level 0 Context DFD

5. DFD Level 1

The Level 1 diagram breaks the main system down into major functional subsystems: Identity, Recommendation, Collaboration, and Mentorship.

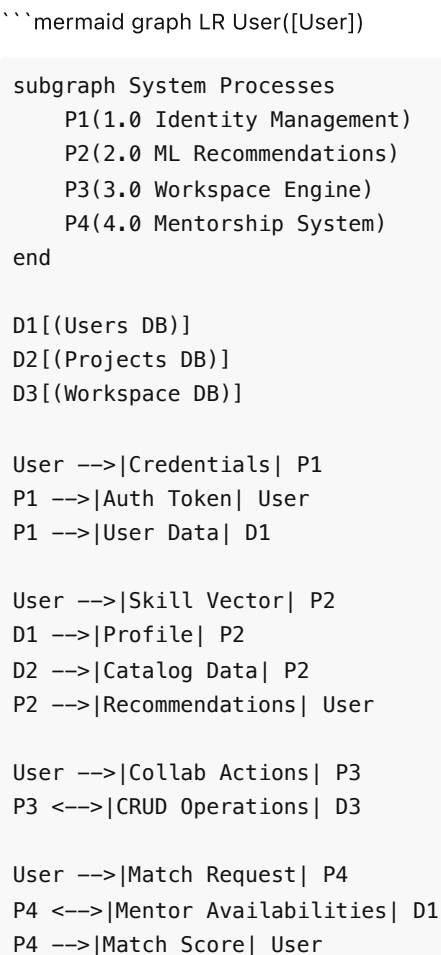


Figure 5: Level 1 Functional DFD

6. DFD Level 2 (Critical Subsystems)

6.1 Workspace & Collaboration Workflow (Level 2)

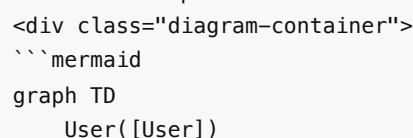
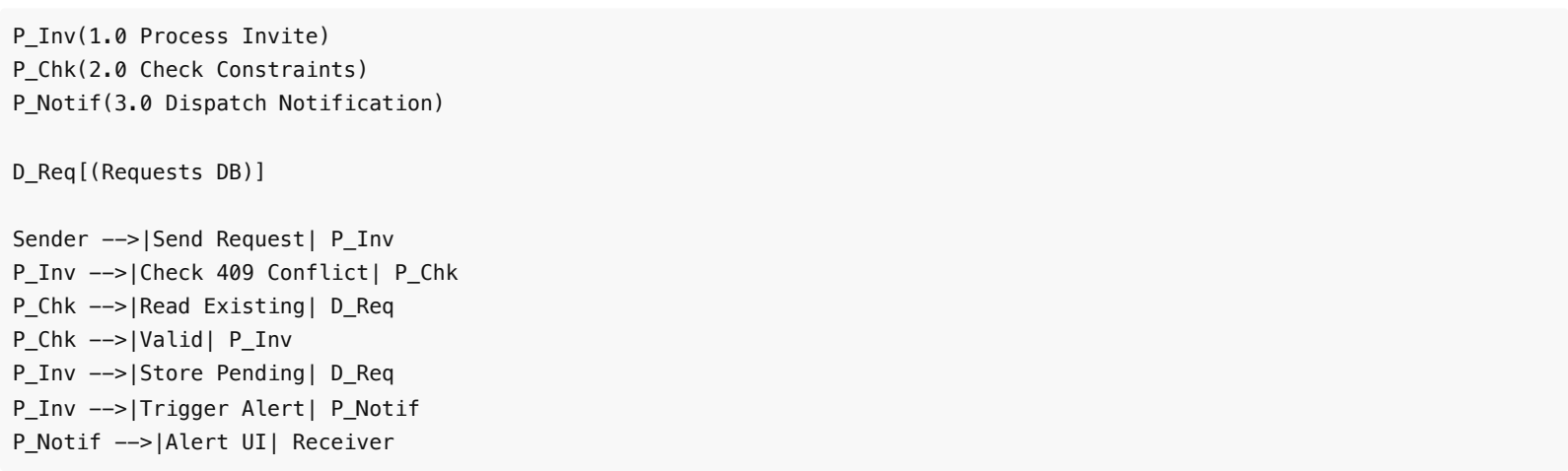




Figure 6.1: Level 2 DFD - Workspace Engine

6.2 Invite & Collaboration Request System (Level 2)

```mermaid graph LR
 Sender([Sender]) --> Receiver([Receiver])



<div class="diagram-caption">Figure 6.2: Level 2 DFD – Smart Invites with 409 Constraint Guard</div>  
</div>

</div>

<div class="page">

<div class="section-label">Section C</div>

<h2>Database / Data Model</h2>

<h3>7. ER Diagram (Logical Data Model)</h3>

<p>ScholarConnect's database architecture is highly relational, utilizing strict foreign key constraints and optimized indexing strategies. The primary entities span across Users, Projects, Skills, Recommendations, and Collaborative Workspaces.</p>

<div class="diagram-container" style="max-height: 800px; overflow-y: auto;">

```mermaid

erDiagram

```
USERS ||--o{ USER_SKILLS : possesses
USERS ||--o{ COLLABORATION_PROJECTS : owns
USERS ||--o{ WORKSPACE_TASKS : assigned
USERS ||--o{ TEAM_MEMBERS : joins
```

```
SKILLS ||--o{ USER_SKILLS : referenced_in
SKILLS ||--o{ PROJECT_SKILLS : referenced_in
```

```
PROJECTS ||--o{ PROJECT_SKILLS : requires
```



Figure 7: Logical Entity-Relationship (ER) Diagram

8. Schema / Collections Model

The SQLite/MySQL schema uses normalized tables to prevent data anomalies. Key collections and their properties include:

| Table / Entity | Primary Purpose | Key Constraints & Relations |
|------------------------|--|---|
| users | Core identity, OAuth tokens, role preferences. | email is UNIQUE. |
| projects | Catalog of base projects from which recommendations are built. | Links to project_skills (M:N). |
| collaboration_projects | User-instantiated projects that are open for teaming. | 1:1 with teams, stores github_repo_url. |

| | | |
|------------------------|--|---|
| collaboration_requests | Tracks invites and join requests (prevents duplication). | UNIQUE constraints across sender and receiver states. |
| mentor_matches | Tracks the lifecycle of a mentorship request. | UNIQUE(user_id, mentor_id) prevents duplicate requests. |
| workspace_tasks | Kanban board persistence. | Tied strictly to team_id. |

9. Schema Design Document

Our schema design adheres to the following principles:

- **Normalization:** M:N relationships (like Users and Skills) are strictly broken down using associative tables (`user_skills` , `project_skills`).
- **Soft Constraints:** Deletions in teams cascade carefully; activity logs maintain historical references.
- **Duplicate Prevention:** The `collaboration_requests` and `mentor_matches` tables utilize composite UNIQUE constraints handled natively by the database engine to guarantee integrity against concurrent race conditions (throwing `409 Conflict`).

Section D

Algorithm / Workflow Diagrams

10. Project Recommendation Workflow

The ML Service handles real-time project recommendation. It processes user skills, academic level, and interests against 20+ diverse domain blueprints to synthesize highly personalized projects.

```
``mermaid flowchart TD
    Start([Client Request]) --> API[Node.js API Gateway]
    API --> ML[ML Service POST /generate]
    ML --> Filter{Filter Blueprints by Domain}
```

```
Filter -->|Match| BP_Match[Select Domain Blueprints]
Filter -->|No Match| BP_Random[Select Random Diverse Blueprints]
```

```
BP_Match --> Synth[Synthesis Engine]
BP_Random --> Synth
```

```
Synth --> Math[Calculate Base Score\nS = (W1 * Skill) + (W2 * Diff) + (W3 * Domain)]
Math --> Randomize[Inject ±15% Random Variance for Diversity]
Randomize --> Sort[Sort by Final Score Descending]
```

```
Sort --> Format[Format JSON Response]
Format --> API
API --> UI([Display to User])
```

<div class="diagram-caption">Figure 8: Project Generation & Recommendation Logic</div></div>

<h3>11. Mentor Matching Workflow</h3>

<p>Matching users to mentors relies on a strictly weighted compatibility formula.</p>

<div class="formula">

Match Score = (0.40 × Skill Match) + (0.30 × Domain Match) + (0.20 × Exp Match) + (0.10 × Availability)

</div>

<div class="diagram-container">

```
``mermaid
```

```
sequenceDiagram
```

```
    participant User
    participant System
    participant Matcher Engine
    participant DB
```

```
User->>System: Request Mentor Match
System->>DB: Fetch User Profile
System->>DB: Fetch Available Mentors
DB-->>System: Mentor Candidates
System->>Matcher Engine: POST /match/mentors (Profile + Candidates)
```

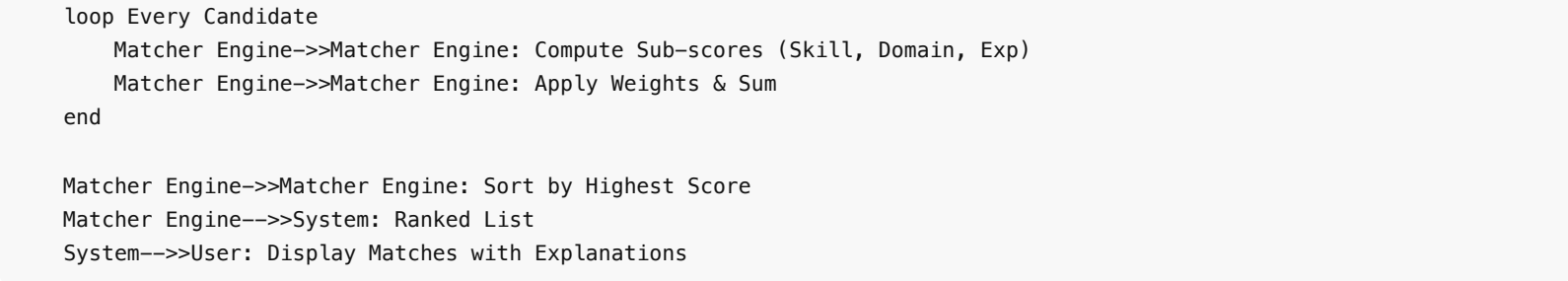


Figure 9: Mentor Ranking & Matching Sequence

12. Smart Invite / Collaboration Workflow

This flow guarantees duplicate prevention via defensive database constraints.

```
``mermaid stateDiagram-v2 [*] --> Idle Idle --> SendingInvite : User Clicks 'Send Invite' SendingInvite --> API : POST /api/collab/requests API --> CheckDB : Check existing records
```

```
state CheckDB {
  direction LR
  Query --> Valid : Not found
  Query --> Conflict : Found Pending/Accepted
}

Conflict --> HTTP409 : Reject Invite
HTTP409 --> UIError : Display "Already Sent"

Valid --> HTTP201 : Insert to DB
HTTP201 --> UISuccess : Update Button State (Disabled)
```

<div class="diagram-caption">Figure 10: State Machine for Duplicate Invite Prevention</div>
</div>

<h3>13. GitHub Import Pipeline Workflow</h3>

```
<div class="diagram-container">
  ``mermaid
  graph LR
    User([User]) -->|Inputs Repo URL| UI[Workspace UI]
    UI -->|PUT /api/workspace/:id/repo| API[Express API]
    API --> DB[(collaboration_projects)]
    API --> Log[(workspace_activity)]
    API --> UI
    UI -->|Render| External[Open GitHub URL in New Tab]
  </div>
```

Figure 11: GitHub Repo Linking Pipeline

Section E

User Flows

14. End-to-End User Journey

The standard user journey for a student moving from onboarding to active collaboration.

```
``mermaid graph TD
  A([Sign Up / Google OAuth]) --> B[Complete Profile & Skills]
  B --> C[View Dashboard Recommendations]
  C --> D[Select Action]
  D -->|Find Project| E[Launch Project]
  D -->|Find Teammates| F[Send Invites]
  D -->|Find Mentor| G[Request Mentor Match]
  E --> H[Create Workspace Team]
  F -->|Accepted| H
  G -->|Accepted| H
  H --> I[Collab Workspace Hub]
  I --> J[Manage Kanban Tasks]
  I --> K[Real-time Chat & Notes]
  I --> L[Link GitHub Repo]
```

<div class="diagram-caption">Figure 12: End-to-end User Journey</div></div>

15. Persona-Specific Flows

Admin Flow

```
``mermaid
graph LR
    Admin([Admin User]) -->|Login| Dashboard[Admin Dashboard]
    Dashboard -->|View Users| Users[User Management]
    Dashboard -->|Review Mentors| Mentors[Approve/Reject Mentors]
    Dashboard -->|System Logs| Logs[View Analytics]
```

Figure 13: Administrator Flow

Section F

Advanced Engineering Diagrams

16. Collaboration Workspace State Machine

The lifecycle of a workspace task through the Kanban system.

```
``mermaid stateDiagram-v2 [*] --> Todo : Task Created

Todo --> InProgress : Start Work
InProgress --> Todo : Blocked/Revert
InProgress --> Done : Complete Task
Done --> InProgress : Reopen Task

Todo --> [*] : Delete
InProgress --> [*] : Delete
Done --> [*] : Delete
```

<div class="diagram-caption">Figure 14: Kanban Task State Machine</div></div>

17. Class Diagram: Core Services

```
``mermaid
classDiagram
    class WorkspaceController {
        +getWorkspace(teamId)
        +createTask(teamId, taskData)
        +updateTask(teamId, taskId, updates)
        +deleteTask(teamId, taskId)
        +updateRepo(teamId, repoUrl)
    }

    class CollabController {
        +inviteUser(req, res)
        +getRequests(req, res)
        +updateRequestStatus(req, res)
    }

    class MLRecommender {
        +generate_synthetic_projects(query, skills)
        +calculate_match_score(weights)
    }

    WorkspaceController ..> CollabController : uses teams
    WorkspaceController --> Database
```



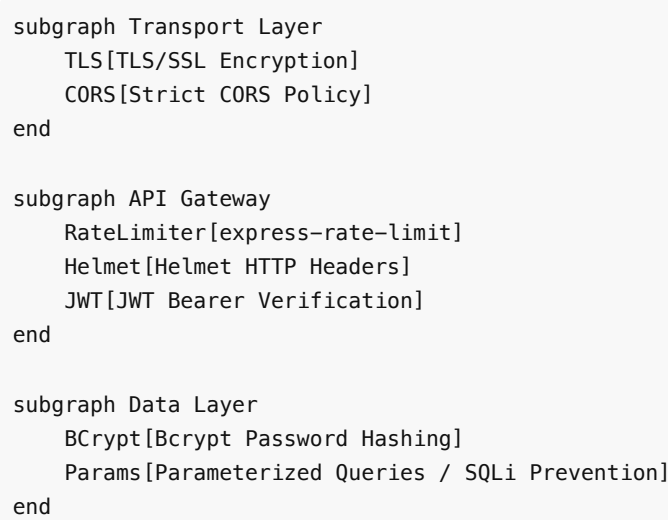
Figure 15: Core Controller Class Diagram
Section G

Security & Scalability

18. Security Architecture

Security is enforced at multiple layers of the application.

```
``mermaid graph TD
    subgraph Client_Layer
        Sanitize[XSS Prevention - React]
        NoStore[No Sensitive Data in LocalStorage]
    end
```



Client Layer --> Transport Layer --> API Gateway --> Data Layer

<div class="diagram-caption">Figure 16: Security Architecture</div>
</div>

- Authentication Flow: JWTs are issued via AuthController and validated using the <code>authenticate</code> middleware on all protected routes.
Data Integrity: Defensive constraints in <code>schema.sql</code> prevent bad state (e.g., duplicated invites, orphaned tasks).

<h3>19. Scalability Architecture</h3>

<p>The system is designed to scale horizontally by decoupling the CPU-intensive Machine Learning operations from the I/O-intensive Express server.</p>

<div class="diagram-container">
``mermaid

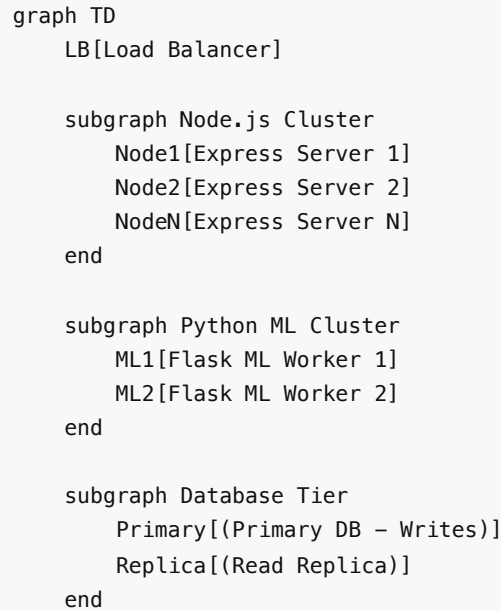




Figure 17: Future Scaling & Microservices Decomposition Diagram

20. Deployment Architecture

The development environment uses Docker Compose to orchestrate the Client, Server, and ML Service. Production deployments target managed container services (e.g., AWS ECS, Google Cloud Run) utilizing the same containerized footprints.